MedievAIl Battle

Simulateur de batailles médiévales en Python
avec IA, terrains à élévation et analyse de Lanchester

Rapport technique

Projet collectif STI3A — INSA Centre-Val de Loire — octobre 2025 à janvier 2026
Rapport rédigé en octobre 2026 à partir du dépôt, par Mohamed Aziz BAOUEB (@aziz3r), mainteneur du miroir

Code source : https://github.com/aziz3r/medievail-battle

Résumé

MedievAIl Battle est un jeu de bataille stratégique en Python (pygame) où deux armées d'unités médiévales — piquiers, chevaliers, arbalétriers et, dans un mode particulier, une merveille à défendre — s'affrontent sur une carte à élévation, pilotées chacune par une intelligence artificielle. Ce document présente l'architecture du code, les règles du modèle de combat, les cinq IA livrées, les interfaces (menu, vue isométrique 2.5D, vue terminal, ligne de commande) et les outils d'analyse (tournois, loi carrée de Lanchester). Il s'appuie sur des mesures effectuées pour l'occasion — un tournoi de 500 matchs et des chronométrages — et signale les limites repérées dans le code. Le projet est une œuvre collective : le dépôt est un miroir et l'historique Git d'origine est conservé.

Table des matières

1 Contexte et objectifs 1

2 Architecture 2

3 Le modèle de jeu 2

4 Les intelligences artificielles 4

5 Interfaces 4

6 Expériences 5

7 Démonstration et reproductibilité 7

8 Limites repérées 7

9 Pistes d'amélioration 7

1 Contexte et objectifs

MedievAIl Battle a été réalisé par une équipe d'étudiants dans le cadre du projet Python / programmation orientée objet de la formation STI3A à l'INSA Centre-Val de Loire. Le dépôt de référence est awa-b/python-project-2025-2026 ; celui-ci en est un miroir maintenu par @aziz3r

(voir `AUTHORS.md` pour la liste des contributeurs et le détail des droits). L'historique Git compte 111 commits d'équipe, du 1er octobre 2025 au 27 janvier 2026.

Les objectifs, tels qu'ils ressortent du code et de la documentation du dépôt, sont les suivants :

— modéliser des unités et un combat de façon orientée objet, avec des règles de dégats chiffrées (bonus de contre, élévation, précision) ;
— fournir plusieurs IA interchangeables, pour pouvoir les comparer entre elles ;
— offrir plusieurs façons d'observer une bataille : interface graphique isométrique, terminal, console ;
— disposer d'outils d'analyse : tournois automatisés et vérification des lois de Lanchester.

Ce rapport complète le dépôt par une description d'ensemble et une évaluation critique. Les chiffres cités sans référence ont été mesurés en octobre 2026 sur un Mac (Python 3.9.6, pygame 2.6.1) ; ils sont indicatifs, car les batailles comportent une part d'aléa (précision des arbalétriers).

2 Architecture

Le code suit un découpage *Modèle / Vue / Présentateur* (MVP). Il représente environ 5 800 lignes de Python (blancs et commentaires inclus) : 1 769 pour le modèle, 1 958 pour la vue, 1 616 pour le présentateur et 497 pour les scripts de lancement à la racine.

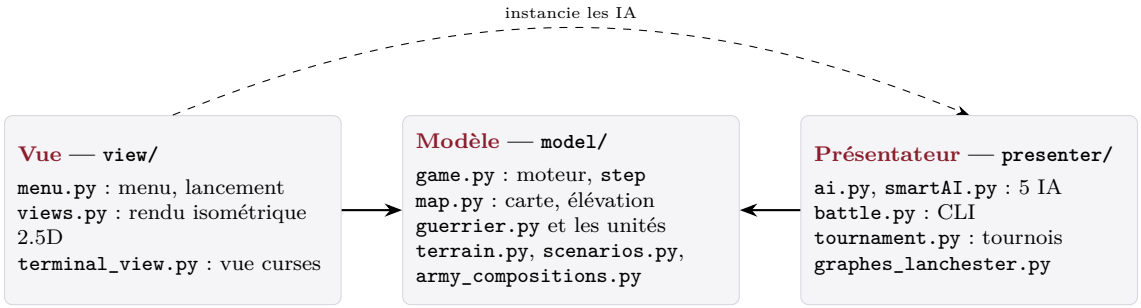


Figure 1 – Organisation du code. La vue lit l'état du modèle et pilote sa boucle ; le présentateur lit l'état et affecte des ordres aux unités ; la vue instancie les IA (flèche pointillée). Points d'entrée : `launch.py`, `run_menu.py`, `visual_simulation.py`, `main.py`, `run_terminal.py`.

Un MVP imparfait. La boucle de simulation (`game.step`) est pilotée depuis la vue : c'est `view/menu.py` (fonction `start_battle_window`) ou `visual_simulation.py` qui l'appelle à chaque image. La vue ne se contente donc pas d'afficher le modèle, elle en cadence l'évolution. Ce choix est simple et efficace pour un jeu, mais il lie le rythme de la simulation à celui de l'affichage ; les tournois contournent le problème avec leur propre boucle sans affichage.

3 Le modèle de jeu

3.1 Carte et unités

La carte (`BattleMap`, 120×120 par défaut) est *continue* : les unités ont des coordonnées flottantes `(x, y)` et la grille ne sert qu'à l'affichage. Seules les bornes sont vérifiées lors d'un déplacement : il n'y a pas de collisions entre unités. L'élévation est une fonction `(x, y)` ↦ altitude fournie par le terrain.

Unité	PV	Attaque	Armure	Portée	Vitesse	Recharge
Chevalier	100	8 (mêlée)	2 / 1	mêlée	1,6	1,8 s
Piquier	55	4 ; +8 contre chevalier	0 / 0	mêlée	1,1	3,0 s
Arbalétrier	35	5 perçante ; +3 contre piquier	0 / 0	7	0,96	2,0 s
Merveille	4 800	—	3 / 10	—	0	—

Table 1 – Caractéristiques des unités (classes `Knight`, `Pikeman`, `Crossbowman`, `Wonder` de `model/`). Armure : normale / perçante. Portée : une valeur nulle signifie mêlée (portée effective d'une tuile). Vision : 4, 4, 9 et 8 tuiles. L'arbalétrier a une précision de 85 % : un tir manqué consomme tout de même la recharge. La merveille est immobile et ne riposte pas.

3.2 Calcul des dégats

Tous les guerriers héritent de la classe abstraite `Guerrier`, qui implémente la formule de dégats du sujet :

$$D \;=\; \max\Big(1,\; k_{\text{élev}} \sum_{i} \max\big(0,\; A_i - R_i\big)\Big)$$

où les A_i sont les composantes d'attaque de l'attaquant (chaque sous-classe en fournit une liste, par exemple l'attaque de base plus un bonus contre un type de cible) et R_i l'armure correspondante de la cible (normale ou perçante). Le coefficient d'élévation vaut $k_{\text{élev}} = 1{,}25$ si l'attaquant est plus haut que sa cible, 0,75 s'il est plus bas et 1 sinon. Deux bonus de contre sont codés dans les unités (piquier contre chevalier, arbalétrier contre piquier) ; les IA s'appuient sur un cycle de type pierre-feuille-ciseaux dont elles ajoutent le dernier côté : le chevalier, rapide, rattrape l'arbalétrier fragile.

Attaquant \ cible	Chevalier	Piquier	Arbalétrier
Chevalier	6	8	8
Piquier	8	4	4
Arbalétrier	4	8	5

Table 2 – Dégats par coup sur terrain plat ($k_{\text{élev}} = 1$), déduits de la formule et des caractéristiques ci-dessus.

3.3 Boucle de simulation

`Game.step(dt)` enchaîne, à chaque pas :

1. pour chaque équipe dont l'intervalle de décision est écoulé, le contrôleur d'IA examine l'état et affecte un *ordre* (`intent`) à ses unités : `("move_to", x, y)` ou `("attack", cible)` ;
2. les cooldowns des unités vivantes diminuent de `dt` ;
3. chaque unité exécute son ordre : elle se déplace vers sa destination, ou poursuit sa cible puis frappe si elle est à portée et si sa recharge est écoulée ;
4. le temps avance et les conditions de victoire sont testées.

La victoire revient à la dernière équipe ayant des unités en vie ; dans le mode *Duel de Merveilles*, elle revient à la dernière équipe dont la merveille est encore debout. Le moteur cumule aussi des statistiques (dégats infligés et subis, victimes, pertes par type, résumé de bataille) et écrit chaque attaque dans un fichier CSV (`battle_attacks.txt`).

3.4 Terrains, compositions et scénarios

Neuf fonctions de terrain définissent l'élévation : plat, colline centrale (*King of the Hill*), deux camps sur collines symétriques, diagonale, vallée centrale, crête horizontale, collines aléatoires (graine fixe), camp ouest avantagé et siège d'un château central. Cinq compositions d'armées se combinent avec ces terrains : *Standard* (27 unités par équipe), *Grande Bataille* (105), *Cavalerie lourde* (85), *Archers massés* (100) et *Formation équilibrée* (100). Six scénarios préconfigurés sont accessibles depuis le menu (Standard, Grande Bataille, Bataille de la Colline, Deux Camps élevés, Siège du Château, Duel de Merveilles), auxquels s'ajoute un scénario N contre $2N$ dédié à Lanchester.

4 Les intelligences artificielles

Cinq contrôleurs héritent de `BaseController` et diffèrent par leur critère de ciblage et par leur fréquence de décision.

IA	Décision	Comportement
Captain BRAINDEAD	0,7 s	Ne se déplace jamais ; n'attaque que les ennemis qui viennent à portée.
Major DAFT	0,3 s	Charge l'ennemi le plus proche puis l'attaque.
Assasin JACK	0,3 s	Vise l'ennemi le plus faible parmi ceux à moins de 15 tuiles, sinon le plus proche.
Predict EINSTEIN	0,15 s	Attribue un score à chaque couple unité–ennemi (coups nécessaires pour tuer, bonus de contre, ennemis blessés, distance) et concentre les tirs (*focus fire*) sans les sur-concentrer.
General STRATEGUS	0,1 s	Règles par type : arbalétriers → piquiers puis arbalétriers ; piquiers → chevaliers ; chevaliers → arbalétriers, avec des seuils de PV pour achever les cibles fragiles.

Table 3 – Les cinq IA et leur intervalle de décision (`decision_interval`).

Remarque. Le menu décrit Predict EINSTEIN comme une IA qui « simule 5 coups à l'avance ». Dans le code lu, l'IA calcule un score instantané pour chaque cible possible et ne simule pas l'état futur du jeu : sa force vient du *focus fire* et des bonus de contre, non d'une recherche en profondeur. La description du menu gagnerait à être mise en conformité.

5 Interfaces

Menu pygame (`view/menu.py`). Il propose cinq entrées : nouvelle bataille, scénarios, chargement d'une sauvegarde, options (vitesse de 10 à 120 images par seconde, lecture automatique) et sortie. La configuration d'une bataille permet de choisir l'IA de chaque camp, la composition et le terrain.

Vue isométrique 2.5D (`view/views.py`). Les unités sont dessinées à partir de planches de sprites animées (repos, marche, attaque, mort, décomposition) dans seize directions ; le terrain utilise un jeu de tuiles. On peut zoomer à la molette (de 0,1 à 2), déplacer la caméra à la souris ou aux flèches, afficher une minimap (`M`), des panneaux de statistiques par armée (`F1`, `F2`, `F3`, `F4`) et faire une sauvegarde / un chargement rapide (`F11`, `F12`, format `pickle`).

Vue terminal (`view/terminal_view.py`), basée sur `curses`, pour observer une bataille sans interface graphique. **Ligne de commande** (`python -m presenter.battle`) : sous-commandes `run`, `load`, `tourney` et `plot`. Un mode console (`main.py`) affiche l'état des armées dans le terminal.



Figure 2 – Parcours du jeu (captures de la démonstration, dossier `demo/`) : menu principal, configuration (IA, composition, terrain), mêlée entre les deux armées et écran de victoire.

6 Expériences

6.1 Tournoi entre les cinq IA

Le module `presenter/tournament.py` joue des matchs sans affichage (pas de 0,2 s, limite de 2 000 pas, au-delà de laquelle le match est nul) et en alternant les camps à chaque manche. Nous l'avons lancé sur le scénario *Standard* (27 unités contre 27), avec 20 manches par confrontation ordonnée, soit $5 \times 5 \times 20 = 500$ matchs, joués en 131 secondes :

```
python -m presenter.battle tourney -G Braindead Daft GeneralStrategus \
       AssasinJack PredictEinstein -S Scenario_Standard -N 20
```

Ligne contre colonne (victoires/20)	Brain.	Daft	Strat.	Jack	Eins.	Total/100
Predict EINSTEIN	20	20	20	20	11	**91**
Major DAFT	20	11	20	17	0	**68**
Assasin JACK	20	3	20	10	0	**53**
General STRATEGUS	20	0	10	0	0	**30**
Captain BRAINDEAD	0	0	0	0	0	**0**

Table 4 – Résultats du tournoi (une exécution, scénario Standard). Les confrontations miroir (même IA des deux côtés) tournent autour de 50 % : 11/20, 10/20, 10/20 et 11/20.

Le classement est transitif : Einstein > Daft > Jack > Strategus > Braindead. Deux observations méritent d'être notées. D'abord, Captain BRAINDEAD n'obtient aucune victoire : immobile, il

ne peut que subir, et contre lui-même aucun combat n'a lieu (résultat cohérent avec des matchs nuls). Ensuite, General STRATEGUS, pourtant l'IA la plus élaborée à règles, perd tous ses matchs contre Daft, Jack et Einstein : dans ce scénario, une charge simple surpasse un ciblage par type. Nous n'avons pas étudié la cause de cet écart ; une hypothèse (non vérifiée) est que la poursuite de cibles spécifiques disperse les unités. Avec 20 manches par case et un aléa sur la précision des arbalétriers non fixé par une graine, les écarts faibles (par exemple Jack contre Jack) ne sont pas significatifs.

6.2 Loi carrée de Lanchester

La loi carrée de Lanchester prédit l'issue d'un combat de masse : si une armée de N unités identiques affronte une armée de $2N$, la plus nombreuse l'emporte et il lui reste $\sqrt{(2N)^2 - N^2} = N\sqrt{3} \approx 1{,}73\,N$ survivants. Le module `presenter/graphes_lanchester.py` vérifie cette prédiction avec le scénario N contre $2N$ (deux IA Major DAFT, N de 2 à 18).

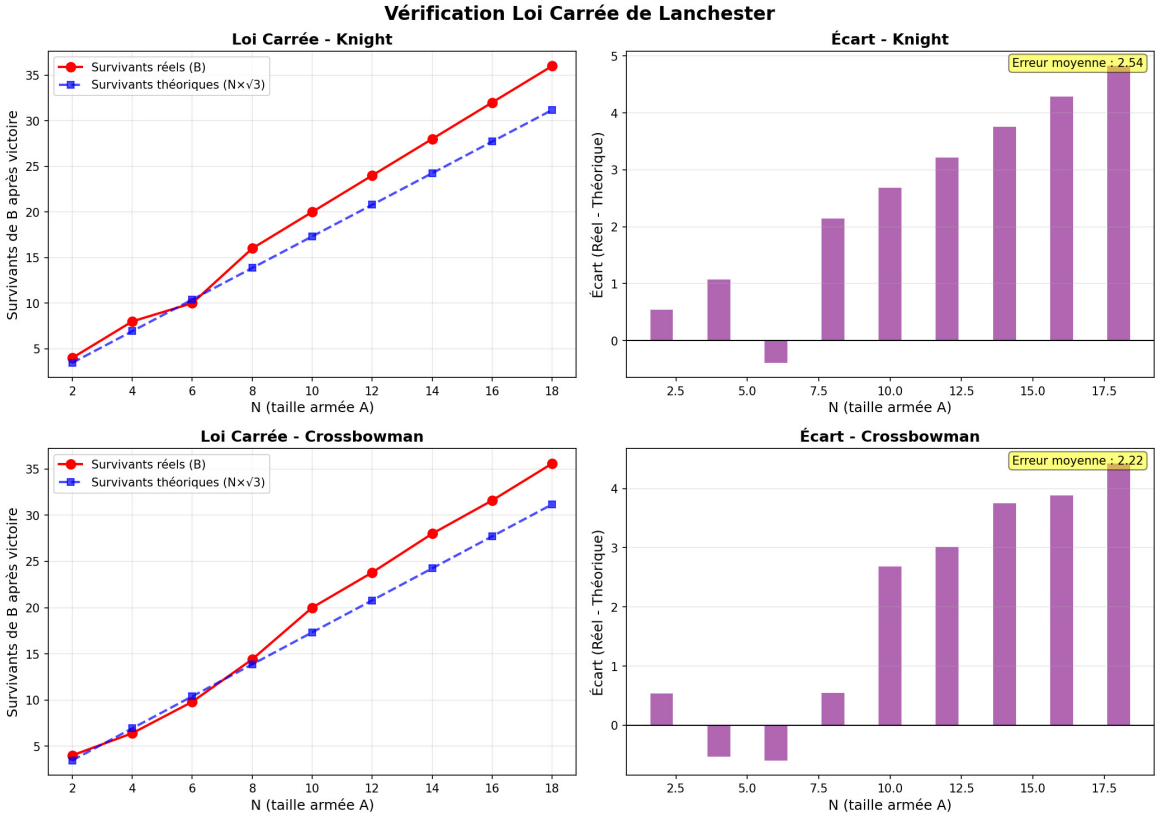


Figure 3 – Vérification de la loi carrée pour les chevaliers et les arbalétriers (fichier `loi_carree_knight_crossbowman.png`, déjà présent dans le dépôt). En rouge, survivants réels du camp B ; en bleu, prédiction $N\sqrt{3}$; à droite, l'écart.

Le résultat ne confirme que partiellement la loi. Le camp le plus nombreux gagne toujours, mais il conserve plus de survivants que prévu : l'écart moyen est de 2,54 survivants pour les chevaliers et de 2,22 pour les arbalétriers, et il augmente avec N. Sur le graphique, le camp B ne perd presque aucune unité pour les chevaliers. Nous n'avons pas analysé la cause de cet écart ; une piste est que le modèle (déploiement en blocs, absence de collisions, ciblage de l'ennemi le plus proche) ne produit pas le combat de masse *simultané* dont la loi carrée suppose l'existence.

6.3 Coût de calcul

Mesurés sans affichage (pas de 0,05 s, terrain *Colline centrale*, General STRATEGUS contre Major DAFT), le scénario Standard (54 unités) se termine en 904 pas et 0,5 s de calcul ; la Grande Bataille (210 unités) en 1 056 pas et 6,8 s. Le coût par pas est donc multiplié par plus de 10 pour un effectif multiplié par 4 : le ciblage parcourt tous les ennemis pour chaque unité, ce qui est quadratique. Avec l'affichage (rendu sans fenêtre), la *Grande Bataille* des scénarios classiques tourne à environ 22 images par seconde.

7 Démonstration et reproductibilité

Le dossier `demo/` contient une vidéo (`demo.mp4`, 55 s), un GIF d'aperçu, onze captures d'écran et le script `capture_demo.py` qui les produit. Celui-ci exécute le vrai jeu (menu, configuration puis bataille jusqu'à la victoire) avec une entrée pilotée par script et un affichage sans fenêtre ; la démonstration illustrée ici oppose General STRATEGUS (A) à Major DAFT (B) en Grande Bataille sur la Colline centrale, et se termine par la victoire de l'équipe B avec 87 survivants.

Pour lancer le jeu lui-même (`COMMENT_LANCER.md` décrit toutes les options) :

```
pip install pygame
python launch.py              # lanceur interactif
python run_menu_windowed.py   # menu fenetre
python -m presenter.battle run Scenario_Standard Daft Braindead
```

8 Limites repérées

Ces points sont issus de la lecture du code ; ils n'enlèvent rien à la réussite d'ensemble du projet.

— **Fichier de journal global.** `Game.__init__` écrase `battle_attacks.txt` dans le répertoire courant et chaque attaque rouvre le fichier en ajout : coût d'entrées-sorties inutile, et impossibilité de jouer deux parties en parallèle dans le même répertoire sans que leurs journaux se mélangent.

— **Deux mécanismes d'ordres.** `Game.apply_actions` sait exécuter des actions retournées par les IA, mais toutes les IA fixent des `intent` et renvoient une liste vide : ce chemin n'est pas utilisé.

— **Option morte du lanceur.** L'entrée 7 de `launch.py` lance `diagnose_menu.py`, fichier absent du dépôt.

— **Aucun test automatisé.** Le dépôt ne contient pas de tests ; la formule de dégats et les conditions de victoire s'y prêtent bien.

— **Reproductibilité.** L'aléa (précision des arbalétriers) n'est pas initialisé par une graine, donc deux tournois identiques peuvent différer légèrement.

— **Performances.** Ciblage quadratique (parcours de tous les ennemis pour chaque unité), voir la section précédente.

— **Documentation de l'IA Einstein.** Voir la section 4.

9 Pistes d'amélioration

1. Ajouter une graine aléatoire paramétrable et des tests unitaires du moteur (dégats, bonus de contre, élévation, victoire).

2. Rendre la journalisation optionnelle et tamponnée, ou la confier à un objet passé au moteur plutôt qu'à un fichier global.
3. Réduire le coût du ciblage (grille spatiale ou recherche du plus proche voisin par cases) pour supporter des armées plus grandes.
4. Ajouter une gestion minimale des collisions, puis rejouer la vérification de Lanchester pour mesurer l'effet sur l'écart à la loi carrée.
5. Si l'on souhaite une IA de recherche, implémenter réellement une simulation de quelques coups ; sinon, corriger la description du menu.
6. Extraire la boucle de simulation de la vue pour se rapprocher d'un MVP strict.